

Improving Performance

In this chapter, you'll review some important points pertaining to the performance of MySQL.

Improving Performance

Database administrators spend a significant portion of their lives tweaking and experimenting to improve DBMS performance. Poorly performing databases (and database queries, for that matter) tend to be the most frequent culprits when diagnosing application sluggishness and performance problems.

What follows is not, by any stretch of the imagination, the last word on MySQL performance. This chapter is intended to review key points made in the previous 29 chapters, as well as to provide a springboard from which to launch discussion and analysis of performance optimization.

So, here goes:

- First and foremost, MySQL (like all other DBMSs) has specific hardware recommendations. Using any old computer as a database server is fine when learning and playing with MySQL. But production servers should adhere to all recommendations.
- As a rule, a critical production DBMS should run on its own dedicated server.
- MySQL is preconfigured with a series of default settings that are usually a good place to start. But after a while, you might need to tweak memory allocation, buffer sizes, and more. (To see the current settings, use `SHOW VARIABLES;` and `SHOW STATUS;`.)
- MySQL is a multi-user multi-threaded DBMS, which means it often performs multiple tasks at the same time. If one of those tasks is executing slowly, all requests will suffer. If you are experiencing unusually poor performance, use `SHOW PROCESSLIST` to display all active processes (along with their thread IDs and execution times). You can also use the `KILL` command to terminate a specific process. (You need to be logged in as an administrator to use `KILL`.)
- There is almost always more than one way to write a `SELECT` statement. Experiment with joins, unions, subqueries, and more to find what is optimum for you and your data.

- Use the `EXPLAIN` statement to have MySQL explain how it will execute a `SELECT` statement.
- As a general rule, stored procedures execute more quickly than individual MySQL statements.
- Use the right data types—always.
- Never retrieve more data than you need. For example, don't use `SELECT *` unless you truly need each and every column.
- Some operations (including `INSERT`) support an optional `DELAYED` keyword that, if used, returns control to the calling application immediately and actually performs the operation as soon as possible.
- When importing data, turn off autocommit. You may also want to drop indexes (including `FULLTEXT` indexes) and then re-create them after the import has completed.
- Database tables must be indexed to improve the performance of data retrieval. Determining what to index is not a trivial task and involves analyzing the `SELECT` statements that have been used to find recurring `WHERE` and `ORDER BY` clauses. If a simple `WHERE` clause is taking too long to return results, you can bet that the column (or columns) being used is a good candidate for indexing.
- Have a series of complex `OR` conditions in your `SELECT` statement? You might see a significant performance improvement by using multiple `SELECT` statements and `UNION` statements to connect them.
- Indexes improve the performance of data retrieval but hurt the performance of data insertion, deletion, and updates. If you have tables that collect data and are not often searched, don't index them until it's really necessary. (Indexes can be added and dropped as needed.)
- `LIKE` is slow. As a general rule, you are better off using `FULLTEXT` rather than `LIKE`.
- Databases are living entities. A well-optimized set of tables might not be so well optimized after a while. As table usage and contents change, so might the ideal optimization and configuration.
- The most important rule is simply that every rule will be broken at some point.

Tip

Browse the Documentation The official MySQL documentation at <http://dev.mysql.com/doc/> is full of useful tips and tricks (and even user-provided comments and feedback). Be sure to check out this invaluable resource.

Summary

In this chapter, you reviewed some important tips and notes pertaining to MySQL performance. Of course, this is just the tip of the iceberg. Now that you have finished reading this book, you are encouraged to experiment and learn as you best see fit.